

Topic Introduction

Welcome to today's PrepLetter! Let's dive into a classic algorithmic tool that's simple, powerful, and shows up everywhere: the **Stack**.

A **stack** is a linear data structure that follows the Last-In, First-Out (LIFO) principle. Imagine a stack of plates: you add (push) a plate to the top, and you remove (pop) the top plate when you need one. Only the top plate is accessible at any time. In programming, stacks are used whenever you need to remember the order of things and process them in reverse.

How does a stack work?

- **Push:** Add an item to the top.
- **Pop:** Remove the top item.
- **Peek:** Look at the top item without removing it.

Why are stacks useful in interviews?

- They help you manage nested or paired structures (parentheses, HTML tags, etc).
- They let you undo operations (think of browser back buttons or text editors).
- They're perfect for parsing, backtracking, and managing function calls.

Simple Example (not one of our problems):

Suppose you want to reverse a string. You could push each character onto a stack, then pop them all off to get the string reversed. This shows how stacks naturally reverse the order of data.

Our Three Stack Problems

Today's trio:

- [Valid Parentheses](#)
- [Backspace String Compare](#)
- [Remove All Adjacent Duplicates In String](#)

Why together?

Each problem is a twist on "should I keep or remove this character based on what came before?" — and stacks let you handle these decisions with elegant, linear solutions. You'll see how pushing and popping based on rules leads to clean, efficient code for tricky string manipulation or matching problems.

Let's start with the most fundamental: matching parentheses.

Problem 1: Valid Parentheses

Problem link: [Valid Parentheses](#)

Rephrased Statement:

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the string is a valid sequence of open and close brackets. The brackets must close in the correct order and type.

PrepLetter: Valid Parentheses and similar

Example:

Input: `s = "{[[]]}"`

Output: `True`

Explanation: Every opening bracket is matched by the correct type of closing bracket in the right order.

How to Think About It:

- When you see an opening bracket, you need to remember it for later.
- When you see a closing bracket, it should match the most recent unclosed opening bracket.
- If at any point the brackets don't match, or if there are leftovers at the end, it's invalid.

Try This Test Case:

`s = "([)]"`

Expected output: `False`

Brute-Force Approach:

Try pairing every closing bracket with an earlier opening bracket, but this is $O(n^2)$ and messy.

Optimal Approach:

Use a stack! Push opening brackets on. When you see a closing bracket:

- If the stack is empty, that's an error.
- If the top of the stack is the matching opener, pop it off.
- If not, it's invalid.

Step-by-step Logic:

- Create an empty stack.
- For each character:
 - If it's an opening bracket, push to stack.
 - If it's a closing bracket:
 - If the stack is empty, return False.
 - If the top of the stack matches, pop.
 - If not, return False.
- At the end, the stack should be empty for a valid string.

Python Solution:

```
def isValid(s):
    # Mapping of closing to opening brackets
    bracket_map = {')': '(', '}': '{', ']': '['}
    stack = []

    for char in s:
        if char in bracket_map.values():
            # Opening bracket: push to stack
            stack.append(char)
        elif char in bracket_map:
            # Closing bracket: check for a match
            if not stack or stack[-1] != bracket_map[char]:
```

```
        return False
    stack.pop()
else:
    # Ignore any non-bracket character (not needed for this problem)
    pass

# Stack should be empty if all brackets matched
return not stack
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$ for the stack

How does this work?

- The `bracket_map` lets us look up the matching opener for each closer.
- We push openers to the stack, and pop them off only if we see a matching closer.
- If we ever mismatch, or the stack is not empty at the end, it's invalid.

Trace Example:

Input: `s = "({[]})"`

Stack: []

- '(': push -> ['(']
- '{': push -> ['(', '{']
- '[': push -> ['(', '{', '[']
- ']': match and pop -> ['(', '{']
- '}': match and pop -> ['(']
- ')': match and pop -> []

Stack is empty. Return True.

Try This Test Case:

`s = "((("`

Expected output: `False` (leftover openers)

Now You Try:

Pause and try to solve this on your own before reading the code again. Can you dry-run the test case above?

Reflect:

Stacks are perfect for this kind of "pairing" problem. Did you know you can also use recursion for similar tasks? Try to think about how that would look!

Problem 2: Remove All Adjacent Duplicates In String

Problem link: [Remove All Adjacent Duplicates In String](#)

Rephrased Statement:

Given a string, repeatedly remove pairs of adjacent duplicate characters until no more can be removed. Return the final string.

Example:

PrepLetter: Valid Parentheses and similar

Input: `s = "abbaca"`

Output: `"ca"`

Explanation:

- Remove "bb" -> "aaca"
- Remove "aa" -> "ca"

How is this similar to Problem 1?

Both use a stack to "undo" or remove previously seen elements based on what comes next. Here, instead of matching pairs (open/close), we look for adjacent duplicates.

Brute-Force Approach:

Scan the string, removing pairs, then repeat until no changes. This can be $O(n^2)$.

Optimal Stack Approach:

Use a stack to keep "safe" characters. If the current character matches the top of the stack, pop it (remove the duplicate pair). Else, push it.

Step-by-step Logic:

- Initialize an empty stack.
- For each character in the string:
 - If stack not empty and `stack[-1] == char`: pop (remove duplicate)
 - Else: push char onto stack
- Join the stack to get the result.

Pseudocode:

```
stack = []
for char in s:
    if stack not empty and stack[-1] == char:
        stack.pop()
    else:
        stack.push(char)
return join(stack)
```

Example:

Input: `s = "abbaca"`

Stack: []

- 'a': push -> ['a']
- 'b': push -> ['a', 'b']
- 'b': pop -> ['a']
- 'a': push -> ['a', 'a']
- 'c': push -> ['a', 'a', 'c']
- 'a': push -> ['a', 'a', 'c', 'a']

Now, at the end, join stack: `'aaca'`

Wait, did we miss a removal? Let's trace carefully:

Actually, after step 4, stack is ['a'] (because after popping the 'b', you push 'a').

Let's try again:

- 'a': push -> ['a']
- 'b': push -> ['a', 'b']
- 'b': pop -> ['a']
- 'a': pop (since 'a' == 'a') -> []
- 'c': push -> ['c']
- 'a': push -> ['c', 'a']

Result: 'ca'

Try This Test Case:

`s = "azxxzy"`

Expected output: 'ay'

Time Complexity: $O(n)$

Space Complexity: $O(n)$ for the stack

Now You Try:

Try solving with pen and paper, or in your code editor. Can you dry-run the above test case?

Problem 3: Backspace String Compare

Problem link: [Backspace String Compare](#)

Rephrased Statement:

Given two strings **S** and **T** which may contain the '#' character, simulate typing them into a text editor. '#' means backspace (delete previous character, if any). Return true if the final strings are equal.

What's new here?

Similar to Problem 2, but now you have two strings and the '#' character acts as a delete. You have to simulate the text editor for both strings, then compare the results.

Brute-Force Approach:

Rebuild each string by simulating the text editor, then compare. If you do this with repeated string concatenation, it's slow ($O(n^2)$).

Optimal Stack Approach:

Use a stack for each string. For each character:

- If it's '#', pop from stack (if not empty).
- Else, push character.

At the end, join the stack for each string, and compare.

Pseudocode:

```
def build(string):
    stack = []
    for char in string:
        if char == '#':
            if stack: stack.pop()
```

```
        else:
            stack.push(char)
        return join(stack)

return build(S) == build(T)
```

Example:

Input: S = "ab#c", T = "ad#c"

Process S:

- 'a': push -> ['a']
- 'b': push -> ['a', 'b']
- '#': pop -> ['a']
- 'c': push -> ['a', 'c']

Process T:

- 'a': push -> ['a']
- 'd': push -> ['a', 'd']
- '#': pop -> ['a']
- 'c': push -> ['a', 'c']

Both become "ac", so output is **True**.

Try This Test Case:

S = "xywrrmp", T = "xywrrmu#p"

Expected output: **True** (since 'u#' cancels out, both end as "xywrrmp")

Time Complexity: $O(n + m)$ (where n, m are lengths of S and T)

Space Complexity: $O(n + m)$ for the stacks

Challenge:

Can you do this with $O(1)$ space? Try thinking about two pointers working backwards!

Summary and Next Steps

You've mastered three classic string manipulation problems using **stacks**:

- Why group them? They all leverage the stack's LIFO nature to build up, undo, or match characters in a linear pass.
- **Key patterns:** Push when you need to "remember," pop when you need to "undo" or "close."
- **Common mistakes:**
 - Forgetting to handle empty stacks (e.g., popping from empty).
 - Not checking for leftovers at the end.
 - Confusing order of operations in popping/pushing.

Variations to watch for:

- Matching different kinds of delimiters (quotes, HTML tags).
- Problems with multi-character "undo" or more complex pairing rules.

- Optimizing for space by modifying in-place or using pointers.

Action List

- Solve all three problems fully on your own — even the one with code provided above.
- Try solving Problem 2 and 3 using a two-pointer approach instead of a stack.
- Explore more stack-based problems (like "Decode String" or "Min Stack").
- Compare your solutions with others to spot different styles or edge-case handling.
- If you get stuck, remember — practice is how you build intuition!

Keep stacking up your knowledge — you've got this!